



Create Kubernetes Manifests

CH Arpita Chowdhury

```
GNU nano 8.3 flask-service.yaml
--
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
```



Introducing Today's Project!

In this project, I will create Kubernetes Deployment and Service manifests to define how my backend application should run and how it should be exposed to users through the EKS cluster. I am doing this because Kubernetes does not deploy applications automatically it relies on manifests to describe how many containers should run, which image to use, and how traffic should reach them. By writing these manifests, I am learning how real production systems declare, manage, and expose cloud-native applications in a scalable and reliable way.

Tools and concepts

I used Amazon EKS, eksctl, Docker, Amazon ECR, and Kubernetes to build and deploy a containerized backend application in a cloud-native environment. eksctl was used to provision the EKS cluster and managed node groups, Docker was used to package the backend into a container image, and ECR stored the image so it could be pulled and run by Kubernetes. Key concepts include using Kubernetes manifests to declare how applications should be deployed and exposed, including defining replicas, container images, labels, and Services. This allowed Kubernetes to automatically create, scale, and manage the backend application in a reliable and repeatable way.



Project reflection

I chose to do this project today because it helped me move beyond infrastructure setup and into actually deploying a real application on Kubernetes, which is one of the most important skills for modern cloud and DevOps roles. Something that would make learning with NextWork even better is having more visual diagrams that show how Deployments, Pods, Services, and ECR connect together, so learners can better visualize how all the components interact inside a Kubernetes cluster.

This project took me approximately 2 hours to complete, including creating the Kubernetes manifests and deploying the backend to the EKS cluster. The most challenging part was understanding how the Deployment and Service manifests work together, especially how labels and selectors connect pods to services and deployments. My favourite part was seeing the backend successfully exposed through Kubernetes and knowing that it was running in a fully cloud-native, scalable environment.



Project Set Up

Kubernetes cluster

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included running this command: `eksctl create cluster \ --name nextwork-eks-cluster \ --nodegroup-name nextwork-nodegroup \ --node-type t3.micro \ --nodes 3 \ --nodes-min 1 \ --nodes-max 3 \ --version 1.33 \ --region us-east-1` in my Ec2 instance terminal I created for this project.

Backend code

I retrieved the backend that I plan to deploy. An app's backend is the server-side part of the application that handles requests, runs business logic, and communicates with data sources. It is responsible for processing user actions and returning the correct responses to the frontend.

Container image

Once I cloned the backend code, I built a container image because Kubernetes requires applications to be packaged into container images before they can be deployed and scaled across a cluster. I used Docker and the project's Dockerfile to build the image, which installed all the required dependencies, copied the backend code into the container, and defined how the Flask application should start when the container runs.

I also pushed the container image to a container registry because Kubernetes cannot use images stored only on a local machine it needs to pull them from a centralized, accessible repository when launching or scaling containers. To push the image to ECR, I created an Amazon ECR repository, authenticated Docker with AWS using the AWS CLI, tagged my local Docker image with the ECR repository URI, and then pushed the image to ECR so it could be securely stored and used by my EKS cluster.



Manifest files

Kubernetes manifests are configuration files, usually written in YAML, that describe the desired state of applications and infrastructure inside a Kubernetes cluster. They tell Kubernetes what to run, how to run it, how many copies to create, and how different components should connect to each other. Manifests are helpful because they make deployments repeatable, predictable, and automated. Instead of manually configuring servers or containers, engineers simply apply a manifest and Kubernetes handles creating, updating, scaling, and healing the application based on those instructions, which is how modern cloud-native systems stay reliable and consistent.

A Kubernetes Deployment manages how your application runs in the cluster by controlling how many container replicas are running, ensuring they stay healthy, and automatically replacing or restarting containers if something fails. The container image URL in my Deployment manifest tells Kubernetes exactly where to find the backend application image in Amazon ECR, so it can pull the correct version of the Docker image and create identical containers across the cluster.



```
GNU nano 8.3 flask-deployment.yaml
--
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest
          ports:
            - containerPort: 8080
```



Service Manifest

A Kubernetes Service exposes a group of running pods through a single, stable network endpoint and load-balances traffic between them. Even if individual pods are created, destroyed, or replaced, the Service ensures that traffic is always routed to healthy instances of the application. You need a Service manifest to make your backend reachable by users or other services. While a Deployment runs the containers, a Service defines how network traffic should enter the cluster and be forwarded to the correct pods, allowing your application to be accessed reliably and consistently.

My Service manifest sets up a NodePort-based network endpoint that exposes the Flask backend on port 8080 and routes incoming traffic to all running pods labeled `app: nextwork-flask-backend`, allowing users and other services to access the application through the EKS cluster.



```
GNU nano 8.3 flask-service.yaml
--
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
```



Deployment Manifest

Annotating the Deployment manifest helped me understand how Kubernetes translates a simple YAML file into real running infrastructure from creating Pods to managing scaling, networking, and self-healing behavior because breaking down each section showed me exactly how labels, selectors, replicas, and container images all work together to tell Kubernetes what to run and how to keep the application healthy in a production environment.

A notable line in the Deployment manifest is the number of replicas, which means how many identical copies of the application Kubernetes should run at the same time. Setting replicas to 3 tells Kubernetes to keep three instances of the backend running so the app can handle more traffic and stay available if one instance fails. Pods are relevant to this because each replica is implemented as a pod in Kubernetes. A pod contains a running container of the application, so when you set replicas to 3, Kubernetes creates and maintains three pods, each running the same backend container.

One part of the Deployment manifest I still want to know more about is the selector and how it matches labels, because it controls which Pods the Deployment manages, and understanding it deeply is important to avoid misconfigurations where a Deployment accidentally controls the wrong Pods or fails to manage the ones it created.



```
GNU nano 8.3 flask-deployment.yaml
apiVersion: apps/v1      # Uses the apps API group (stable) for Deployments in Kubernetes
kind: Deployment          # A Deployment manages Pods for you (creates, replaces, scales, rolls out updates)

metadata:                # Identification info for this Deployment object
  name: nextwork-flask-backend # The unique name of this Deployment in the cluster (used in kubectl commands)
  namespace: default          # The Kubernetes namespace where this Deployment will live (default = main workspace)

spec:                    # The "desired state" instructions Kubernetes should enforce for this Deployment
  replicas: 3             # Keep 3 identical Pods running for high availability + load sharing (self-healing too)

  selector:              # How the Deployment finds which Pods belong to it
    matchLabels:
      app: nextwork-flask-backend # Deployment manages ONLY Pods with this exact label (must match template labels)

  template:              # Blueprint for the Pods that will be created by this Deployment
    metadata:
      labels:
        app: nextwork-flask-backend # Label applied to each Pod so the Deployment + Service can target it

    spec:                # Pod specification: what runs inside each Pod
      containers:
        - name: nextwork-flask-backend # Container name inside the Pod (useful for logs/exec if multiple containers)
          image: 006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest
          # The exact Docker image to pull from Amazon ECR (latest tag). Kubernetes pulls this to create containers.

      ports:
        - containerPort: 8080          # The port the Flask app listens on inside the container (internal Pod port)
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

